

Create & Display Views

In Laravel, the view layer is responsible for handling the presentation of data to the user, typically in the form of HTML. Laravel's view system allows developers to separate the logic of the application from the display of the data, adhering to the Model-View-Controller (MVC) design pattern.

The view is where you define the user interface of your application. Laravel provides a powerful templating engine called Blade, which allows you to easily create reusable components, layouts, and dynamic content.

Using A Frontend Framework

Another very common use case for views is to use a frontend framework like Vue.js or React.js. You can use Laravel to serve the API and then use a frontend framework to consume the API and render the HTML. This is a very common pattern in modern web development. This makes your UI more interactive and responsive. Although you can certainly have interactive elements just by using either vanilla JavaScript or something like Alpine.js, which we're going to be using later.

Creating Views

To create a view in Laravel, you can use the `view()` helper function. The `view()` function takes the name of the view as its first argument. You can also pass data to the view but we'll look at that in the next lesson.

Ultimately, we will be using Blade templates, but views do not have to be Blade templates. They can be plain PHP files as well so that is what we will start with.

Let's create a new file at `resources/views/jobs.php` with the following content:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Jobs List</title>
  </head>
  <body>
    <h1>Available Jobs</h1>
    <ul>
      <li>Web Developer</li>
      <li>Software Engineer</li>
      <li>System Analyst</li>
      <li>Database Administrator</li>
    </ul>
  </body>
</html>
```

It is just simple HTML for now. Of course, you can also use PHP in this file. You just need to use the `<?php ?>` tags. For example, you can replace the `h1` tag with the following PHP code:

```
<h1><?php echo 'Available Jobs' ?></h1>
```

Now, let's update our route to return this view:

```
Route::get('/jobs', function () {  
    return view('jobs');  
});
```

Now, when you visit `/jobs` in your browser, you should see the HTML.

Sub Folders For Views

You can also create subfolders in the `resources/views` directory to organize your views. For example, let's create a folder named `jobs` and rename the `jobs.php` file to `index.php` and move it to that folder.

Now, update the route to return the view from the subfolder:

```
Route::get('/jobs', function () {  
    return view('jobs.index');  
});
```

We can use dot notation to specify the subfolder and the view file. This is a common pattern in Laravel. So now, if we have let's say a view to add a new job, we can create a file named `create.php` in the `jobs` folder and return it like this:

```
Route::get('/jobs/create', function () {  
    return view('jobs.create');  
});
```

In the next lesson, I will show you how to pass data to views.